



Instituto Politécnico Nacional

Escuela Superior de Cómputo

INGENIERÍA EN SISTEMAS COMPUTACIONALES

Lab 13.

Schoolbook RSA

Corona Hernández Martín Rafael

6CV3

Introduction to Cryptography

June 17, 2026

FUNCTIONS

```
1 def generar_claves_rsa(bits):
2     """Genera el par de claves RSA (pública y privada)"""
3     p = getPrime(bits)
4     q = getPrime(bits)
5     while p == q:
6         q = getPrime(bits)
7
8     n = p * q
9     phi_n = (p - 1) * (q - 1)
10
11     while True:
12         e = random.randrange(2, phi_n)
13         if e not in (3, 65537) and math.gcd(e, phi_n) == 1:
14             break
15
16     d = pow(e, -1, phi_n)
17     return (e, n), d
```

```
1 def menu_generar_claves():
2     print("\n--- 1. Generación de Claves RSA ---")
3     bits = int(input("Ingresa el tamaño de los primos en bits (32, 512, 2048): "))
4     public_key, private_d = generar_claves_rsa(bits)
5
6     print(f"\n[+] Clave Pública generada: (e={public_key[0]}, n={public_key[1]})")
7     print(f"[+] Exponente Privado d generado: {private_d}")
8
9     guardar_claves_en_archivos(public_key, private_d)
```

The program generates a valid RSA key pair by selecting two prime numbers of a specified bit length. It explicitly ensures that the public exponent e is valid ($e \notin \{3, 65537\}$ and $\gcd(e, \phi(n)) = 1$) and calculates the private exponent d . The keys are exported to text files for easy sharing.

```

1 def cifrar_rsa_desde_archivo():
2     print("\n--- 2. Cifrar Mensaje (TBC16) desde Archivo ---")
3     archivo_pub = input("Nombre del archivo de clave publica a usar (ej. 'publica.txt'): ").strip()
4
5     try:
6         with open(archivo_pub, "r") as f:
7             lineas = f.read().splitlines()
8             e = int(lineas[0])
9             n = int(lineas[1])
10            print(f"[+] Clave pública cargada correctamente de {archivo_pub}")
11    except FileNotFoundError:
12        print(f"[-] Error: No se encontró el archivo '{archivo_pub}'.")
13        return
14    except Exception as err:
15        print(f"[-] Error al procesar el archivo: {err}")
16        return
17
18    # Generar r aleatorio de 16 bits
19    r = random.getrandbits(16)
20    print(f"Número aleatorio de 16 bits generado (r): {r}")
21
22    # Cifrar usando la clave pública: c = r^e mod n
23    c = pow(r, e, n)
24    print(f"Texto cifrado resultante (c): {c}")
25
26    # Guardar el texto cifrado en un archivo para compartirlo fácilmente
27    with open("cifrado.txt", "w") as f:
28        f.write(str(c))
29    print("[+] El texto cifrado se ha exportado a 'cifrado.txt'")

```

Encryption: It reads a partner's public key from a text file, generates a random 16-bit integer r to simulate a symmetric key, and encrypts it using the formula $c = r^e \bmod n$. The resulting ciphertext is saved to a file.

```

1 def descifrar_rsa_desde_archivo():
2     print("\n--- 3. Descifrar Mensaje desde Archivo ---")
3     archivo_priv = input("Nombre del archivo de tu clave privada (por defecto 'privada.txt'): ").strip()
4
5     try:
6         with open(archivo_priv, "r") as f:
7             lineas = f.read().splitlines()
8             d = int(lineas[0])
9             n = int(lineas[1])
10            print(f"[+] Clave privada cargada correctamente de {archivo_priv}")
11    except FileNotFoundError:
12        print(f"[-] Error: No se encontró el archivo '{archivo_priv}'.")
13        return
14    except Exception as err:
15        print(f"[-] Error al procesar el archivo: {err}")
16        return
17
18    opcion_c = input("¿Deseas leer el texto cifrado 'c' desde un archivo .txt? (s/n): ").strip().lower()
19    if opcion_c == 's':
20        archivo_c = input("Nombre del archivo cifrado (ej. 'cifrado.txt'): ").strip()
21        try:
22            with open(archivo_c, "r") as f:
23                c = int(f.read().strip())
24            print(f"[+] Texto cifrado cargado: c = {c}")
25        except Exception as err:
26            print(f"[-] Error al leer el archivo cifrado: {err}")
27        return
28    else:
29        c = int(input("Ingresa el valor de 'c' manualmente: "))
30
31    # Descifrar usando la clave privada: m = c^d mod n
32    m = pow(c, d, n)
33    print(f"\n[+] Mensaje original recuperado exitosamente (m): {m}\n")
34

```

Decryption: It reads the user's private key and the ciphertext from their respective files, applying the formula $m = c^d \bmod n$ to successfully recover the original 16-bit message.

SCREENSHOTS CODE RUNNING

```
--- 1.1 Números Primos Aleatorios ---
Primo de 16 bits: 38693
Primo de 32 bits: 3091546637
Primo de 512 bits: 12330116434260770125960679948856389399654064853288998209969326520788010385179605298319525759065095
781636220949852766008121486250964943688884045631459599053
Primo de 2048 bits: 2844573026400817504480597269230361586960467775219512475803274520014481587514684483525474474966381
278332186819118439423233775796121877788223856463336717420723090832523009534472354495352860010472715121995983964003776
424613380884522863414354874451869063594773626337702820844828756417292938897231229883191589178898135112553065743529293
23313337339546130454945484921971567128449818316462286481650979003339144735562475182718136002089307237123207568618878
342205259887881023618743971794864747789347814067544684520040851874789049003538794155215250027534401839982463356991315
5815496792280828013531715791053609330171874748510841
```

KEYS GENERATION

```
O martincorona@MacBook-Pro-de-Martin LAB13 % python3 rsa.py
===== LAB 13: RSA CON PERSISTENCIA EN TEXTO =====
1. Generar llaves y guardarlas en archivos (.txt)
2. Cifrar leyendo clave pública desde un archivo
3. Descifrar leyendo clave privada desde un archivo
4. Salir
Selecciona una opción: 1

--- 1. Generación de Claves RSA ---
Ingresa el tamaño de los primos en bits (32, 512, 2048): 512

[+] Clave Pública generada: (e=74514272813158903641779016412558834717533637964315649234677145157531502124179338072941
048463282561768985066331243032974788869040944137959754265365868889142378919504251579271809085869422057614767359219594
915146568433945295170005233450095955762942124929409002847743146975307200528747499775407895506490387738773, n=78854805
579466548718638794525448536830463266385315809458949831167567742037821797795757968110757774387184417528442335209628037
080052728877139704562158139842873455321847528995012320914842259757682099134949627373436193757434424776025749403012680
565146803994821052526267781824676789277170243627459717415946250107)
[+] Exponente Privado d generado: 84632696516955069630607677163620897983104444954109105986083738012482007603352879442
300992538726276099856159070314487970706139387354429160376072266455165096261421632721322511449984188495232691307461308
87085129901585457967560027825291204189585978865720875820428541746304384339065965569887668807067672726578477

[+] Archivos de texto generados exitosamente:
-> 'publica.txt' (Contiene e y n)
-> 'privada.txt' (Contiene d y n)
```

CIPHER

```
===== LAB 13: RSA CON PERSISTENCIA EN TEXTO =====
1. Generar llaves y guardarlas en archivos (.txt)
2. Cifrar leyendo clave pública desde un archivo
3. Descifrar leyendo clave privada desde un archivo
4. Salir
Selecciona una opción: 2

--- 2. Cifrar Mensaje (TBC16) desde Archivo ---
Nombre del archivo de clave pública a usar (ej. 'publica.txt'): publica.txt
[+] Clave pública cargada correctamente de publica.txt
Número aleatorio de 16 bits generado (r): 17483
Texto cifrado resultante (c): 105763029090882799780029307319925128169737253795905332454959514138732664829129507951884
50896902339765997740525433484521134358920852067483320076113010570489672652151022828253335789171354202210869573636579
38735488153081949168510401412001521718832788567290311327890045727643932466178943955697541867431957587530
[+] El texto cifrado se ha exportado a 'cifrado.txt'
```

DECIPHER

```
===== LAB 13: RSA CON PERSISTENCIA EN TEXTO =====
1. Generar llaves y guardarlas en archivos (.txt)
2. Cifrar leyendo clave pública desde un archivo
3. Descifrar leyendo clave privada desde un archivo
4. Salir
Selecciona una opción: 3

--- 3. Descifrar Mensaje desde Archivo ---
Nombre del archivo de tu clave privada (por defecto 'privada.txt'): privada.txt
[+] Clave privada cargada correctamente de privada.txt
¿Deseas leer el texto cifrado 'c' desde un archivo .txt? (s/n): s
Nombre del archivo cifrado (ej. 'cifrado.txt'): cifrado.txt
[+] Texto cifrado cargado: c = 10576302909088279978002930731992512816973725379590533245495951413873266482912950795188
45089690233976599774052543348452113435892085206748332007611301057048967265215102282825333578917135420221086957363657
938735488153081949168510401412001521718832788567290311327890045727643932466178943955697541867431957587530

[+] Mensaje original recuperado exitosamente (m): 17483
```